

Entwicklung eines Fragentyps für Java-Codeaufgaben in H5P

Mohammed Schergis

Bachelorarbeit

Vorgelegt am Lehrstuhl für Rechnernetze.

Abgabe der Arbeit: Juni 2025
Erstgutachten: Dr. Markus Brenneis
Zweitgutachten: Dr. John Witulski

Zusammenfassung

Interaktive Wissenstests in Lernmanagementsystemen wie Ilias profitieren insbesondere von speziell zugeschnittenen Aufgabentypen, wo Programmierkompetenzen geprüft werden sollen. Bisher fehlte in H5P ein Fragetyp, der Codeaufgaben mit Syntax-Highlighting, Lückentext-Mechanismen und einer prüfbaren Code-Auswertung kombiniert. Ziel dieser Arbeit war es daher, einen solchen Fragetyp zu entwerfen und umzusetzen, zugleich aber auch die vielfach undokumentierten Eigenheiten des H5P-Ökosystems systematisch aufzubereiten.

Ausgehend von einer Analyse der H5P-Spezifikation wurden zentrale Limitierungen identifiziert: veraltete ES5-Randbedingungen, und verstreute Community-Guides statt offizieller Dokumentation. Um moderne ECMAScript-Features dennoch nutzen zu können und moderne JavaScript-Bibliotheken zu integrieren, wird ein Webpack-gestützter Build-Prozess vorgestellt, der Quellcode über Babel transpiliert und damit vollständig H5P-kompatibel macht.

Das Ergebnis ist ein lauffähiger H5P-Fragetyp, der Lückentexte in Java-Code-Snippets unterstützt, Nutzereingaben prüft und sofortiges Feedback liefert. Die Arbeit deckt die ursprüngliche Problemstellung teilweise ab und bildet zugleich eine Basis für weiterführende Funktionalitäten, wie clientseitige Code-Evaluation für nicht-browser-nativer Sprachen. Neben dem funktionalen Artefakt leistet die Arbeit eine systematische Aufbereitung der H5P-Konventionen, identifizierter Stolpersteine und empfohlene Workarounds. Die gewonnenen Erkenntnisse und bereitgestellten Werkzeuge reduzieren die Einstiegshürden künftiger Projekte signifikant und bilden einen Ausgangspunkt für vertiefende Forschung zu automatisierten Programmiertests im E-Learning-Kontext.

Inhaltsverzeichnis

Abbildungsverzeichnis	v
Tabellenverzeichnis	vi
Listings	vii
1 Einleitung	1
1.1 Motivation	1
1.2 Aufbau der Arbeit	2
2 Details zu H5P	3
2.1 H5P	3
2.2 H5P-Bibliotheken	5
3 H5P Library Entwicklung	7
3.1 H5P Entwicklungsumgebung	7
3.1.1 H5P-CLI fehlende Dokumentation und Probleme	8
3.1.2 Representation von anderen Plattformen	9
3.2 H5P Dateiformat	9
3.2.1 Library.json	11
3.2.2 Semantics.json	12
3.2.3 H5P-spezifische Vorgaben und Limitierungen für JavaScript	13
3.3 Nutzung von anderen H5P-Bibliotheken als Abhängigkeit	16
4 Nutzung von Package Managern	19
4.1 Webpack und Babel	20
4.2 Einrichten eines Buildprozesses für H5P	21

5	Entwickelte H5P-Bibliotheken	25
5.1	H5P.SyntaxHighlighting	26
5.2	H5P.Blanks	27
6	Java Code Kompilierung und Interpretation in H5P	29
7	Fazit	31
7.1	Related Works	31
7.2	Diskussion der Ergebnisse	31
	Literatur	33

Abbildungsverzeichnis

3.1	Typische Ordnerstruktur einer H5P-Bibliothek	10
5.1	Entwickelter Prototyp 'H5P.SyntaxHighlighting'	26
5.2	Entwickelter H5P-Fragetyp 'H5P.SyntaxBlanks'	28

Tabellenverzeichnis

3.1	Erforderliche Felder in der 'library.json' Datei	11
-----	--	----

Listings

3.1	H5P Setup-Kommand	8
3.2	H5P Utils-Pack-Kommand	8
3.3	Check Funktion in der Code Base von h5p-cli	10
3.4	H5P Helper Klasse in IIFE Kapselung	15
3.5	H5P Klasse in IIFE Kapselung	15
3.6	H5P Prototyp Erweiterung in IIFE Kapselung	18
4.1	Ordnerstruktur für H5P mit Webpack und Babel	23
4.2	Watch Skript für das Development Environment	23
4.3	Build Skript für das Production Environment	23
4.4	Template für die Webpack Konfiguration	24
4.5	Template für die Babel Konfiguration	24

Kapitel 1

Einleitung

1.1 Motivation

In Lehreinrichtungen werden Wissenstests für Lernmanagementsysteme wie Moodle eingesetzt, um den Wissensstand der Lernenden zu evaluieren und zu konsolidieren. Aus didaktischer Perspektive wäre es für Lehrende, die Module betreuen und Code-Kompetenzen vermitteln, empfehlenswert, Code-Aufgaben zu stellen, die durch Syntax-Highlighting und/oder eine direkte Code-Auswertung unterstützt werden. Dies resultiert in einer erhöhten Zugänglichkeit und Attraktivität für die Lernenden.

H5P ist ein Open-Source-Plugin für Content-Management-Plattformen, das die Erstellung und den Austausch interaktiver Webinhalte ermöglicht. In der Regel findet sie in Bildungseinrichtungen Anwendung als didaktisches Instrument, um Lerninhalte auf ansprechendere und interaktivere Weise zu gestalten. H5P unterstützt eine Vielzahl von Content-Management-Plattformen, die für Lernzwecke eingesetzt werden, wie etwa Moodle und Ilias. Die vorliegende Untersuchung widmet sich der Entwicklung eines neuartigen Fragentyps für Java-Codeaufgaben in H5P. Das Ziel dieser Entwicklung ist die Erweiterung der Möglichkeiten der Plattform sowie die Bereitstellung einer effizienteren Methode zur Testung und Verbesserung von Programmierkenntnissen für Lernende.

Diese Arbeit beschäftigt sich mit dem Entwickeln von Bibliotheken für H5P. Die mangelnde und sporadisch stattfindende Dokumentation der Spezifikationen und Konventionen von H5P stellt ein Hindernis für Entwickler dar, die sich mit H5P-Bibliotheken beschäftigen möchten. Die Arbeit mit H5P legt die Limitierungen und Herausforderungen, die im Entwicklungsprozess von H5P-

Bibliotheken evident werden offen. In der Folge widmen wir uns der Fragestellung, welche Aspekte bei der Einrichtung einer Entwicklungsumgebung für H5P-Bibliotheken zu berücksichtigen sind, um eine ideale Umgebung zu schaffen.

Die Nutzung einschlägiger Bibliotheken ist ein essenzieller Bestandteil eines Softwareprojekts. Die Kombination mit dem H5P-Framework erweist sich als eine nicht trivial zu bewältigende Aufgabe. In der vorliegenden Arbeit werden die Herausforderungen und Lösungen diskutiert, die im Zusammenhang mit der Entwicklung eines neuen Fragentyps unter Verwendung von JavaScript-Paketen entstehen.

1.2 Aufbau der Arbeit

Zunächst wird ein grundlegender Überblick über H5P und seine Rolle als Open-Source-Framework zur Erstellung interaktiver Lerninhalte gegeben. Im Anschluss erfolgt eine systematische Aufarbeitung der relevanten, häufig verstreuten Spezifikation. Aufbauend auf diesen Erkenntnissen wird dargelegt, wie sich zeitgemäße JavaScript-Bibliotheken unter Zuhilfenahme von Buildprozessen in ein kompaktes Format überführen lassen, welches von H5P erwartet wird. Im weiteren Verlauf erfolgt die Präsentation der eigens entwickelten Fragentypen in H5P. Diese dienen als praktische Beispiele sowie zur Validierung der beschriebenen Vorgehensweisen. Abschließend werden mögliche Lösungen präsentiert, mit deren Hilfe Code Kompilierung und Interpretation im Client ermöglicht werden und daraus mögliche Fragentypen in H5P.

Kapitel 2

Details zu H5P

In diesem Abschnitt werden wir uns anschauen was genau H5P ist und was es versucht zu lösen. Dabei werden wir uns einen Überblick verschaffen von H5P, den Bausteinen des Frameworks, und die Rolle vom H5P-Plugin.

2.1 H5P

H5P steht für 'HTML 5 Package' und ist ein Plugin für Content-Management-Plattformen (CMS) oder Learning-Management-Systeme (LMS). H5P ist ein Open-Source-Framework, das die Erstellung und den Austausch interaktiver Inhalte (bezeichnet als Content Typen) ermöglicht, welche in diversen Content-Management-Systemen (CMS) verwendet werden können. Die Integration spezifischer Content-Typen in Content-Management- bzw. Learning-Management-Systeme erlaubt es den Autorinnen und Autoren, Inhalte dieser spezifischen Content-Typen zu generieren. Dies erfolgt über Konfigurations- und Inhaltsfelder, die über Schnittstellen wie beispielsweise einen WYSIWYG-Editor bereitgestellt werden. Die erstellten Inhalte (Content) können auch wieder exportiert und in andere Systeme importiert werden, wobei eine exakte Repräsentation und Funktionalität gewährleistet ist. [H5P17, Automatically becomes importable and exportable]

H5P erleichtert diese Erstellung von Lerninhalten und stellt die dafür erforderliche Infrastruktur bereit, um eigenen Logik auszuführen und dynamische Elemente auf Content-Management-Systemen (CMS) darzustellen, die für statische Inhalte konzipiert sind und ohne den Einsatz entsprechender Plugins dies nicht ermöglichen.

H5P bietet das nötige Interface für den Import und Export von Inhalten. Voraussetzung ist eine H5P-Integration in Form eines Plugins für das jeweilige CMS/LMS. Entwickler von Content-Typen müssen sich nicht um den Support für verschiedene Plattformen kümmern. Das H5P Core Team bzw. der Maintainer der Integration übernimmt den Support in diesen Systemen. [H5P17, Platform(CMS/LMS) agnostic] Eine Liste der unterstützten Integrationen als Plugin, die vom Core-Team von H5P unterstützt werden, ist:

- WordPress
- Drupal
- moodle

Es gibt auch Integrationen von Plugins für andere Systeme wie Ilias, die von sr.solutions entwickelt wurde und maintained wird. [AG25]

Eine H5P Integration als Plugin kann man sich wie ein Interface vorstellen. Ein Teil davon ist der Core, der sich um Persistenz und Verwaltung von Bibliotheken, um Feature-Updates zu Bibliotheken, um die Validierung von Content einschließlich dessen Bereinigung, um das Speichern und Laden von Content sowie um die Bereitstellung von APIs kümmert. [H5P25g, The H5P Core] Ein weiterer Teil ist der Editor, der Bearbeitungs- und Autoren-Features für H5P bereitstellt. Er liest die Datenstruktur und Semantik aus der Konfigurationsdatei der H5P-Bibliotheken und generiert anhand dieser Informationen ein Formular für Autoren. Der Editor stellt Standard-Widgets für alle Datentypen bereit, die von 'custom widgets' überschrieben werden können. So ist es beispielsweise möglich, einen „What You See Is What You Get“-Editor (WYSIWYG) bereitzustellen. [H5P25g, The H5P Editor]

Alternative können H5P Content-Typen auch auf H5P.com erstellt, gehostet und über einen Link auf beliebigen Webseiten eingebettet werden, oder über LTI (Learning Tools Interoperability) in andere Systeme integriert werden. Hierauf wird in dieser Arbeit nicht weiter eingegangen.

Dabei verspricht H5P Entwicklern, mehr in weniger Zeit zu schaffen. [H5P17, Why develop for H5P?] Dies soll durch den modularen Aufbau von H5P und die Vielzahl bereits bestehender Content-Typen möglich sein. Diese können als Bausteine verwendet werden, um neue Content-Typen zu erstellen. Dies sorgt für konsistentes Verhalten bei ähnlichen Content-Typen, da Entwickler nur die contenttyp-spezifische Logik und Oberfläche implementieren müssen. Das Erstellen des Autoren Interfaces ist hierbei lediglich eine Konfigurationsdatei im JSON Format. Entwickler

von Content-Typen müssen nur die nötigen Daten beschreiben und dessen Struktur. Das Autoren Interface sowie die Datenbereinigung werden aus den definierten Daten generiert von H5P. [H5P17, Creating the editor is extremely efficient]

2.2 H5P-Bibliotheken

H5P-Bibliotheken (H5P-Library genannt) sind JavaScript-Bibliotheken, die einige H5P-spezifische Konfigurationsdateien im JSON-Format sowie Assets wie Bilder und CSS-Dateien enthalten. H5P-Bibliotheken sind die zentralen Bausteine von H5P, die in drei Typen unterteilt werden können [H5P25g, The H5P Libraries]:

- Content-Typ Bibliotheken
- Editor Bibliotheken
- API Bibliotheken

Content-Typ Bibliotheken sind die eigentlichen H5P-Bibliotheken, die von Autoren verwendet werden um Inhalte zu erstellen. Sie sind die einzige Art von H5P-Bibliotheken, die sich selbstständig im bereitgestellten Bereich der Webseite darstellen können. Sie bestehen typischerweise aus mehreren H5P-Bibliotheken mit Abhängigkeiten untereinander. Editor Bibliotheken sind H5P-Bibliotheken, die 'custom widgets' bereitstellen, wie erwähnt einen WYSIWYG-Editor. API Bibliotheken stellen Hilfsfunktionen für andere H5P-Bibliotheken bereit. Ein Beispiel hierfür ist 'H5P.Question', das eine einheitliche Logik und Darstellung für Fragetypen bereitstellt.

Alle Arten von H5P-Bibliotheken benötigen mindestens eine JSON Konfigurationsdatei mit dem Namen 'library.json'. Die darin definierten Metadaten dienen vor allem zur Identifikation der Bibliothek und ihrer Abhängigkeiten zu anderen Bibliotheken. Außerdem benötigt jede H5P-Bibliothek eine JavaScript Datei, die den Code enthält. Für eine Editor oder API Bibliothek sind das die nötigen Dateien die H5P erwartet. Für eine Content-Typ-Bibliothek muss zudem eine Konfigurationsdatei namens 'semantics.json' vorhanden sein. Diese definiert die Semantik und Struktur der Daten, die zum Generieren des Autoren-Interfaces verwendet werden. Hier können Standard-Widgets wie Text- oder Auswahlfelder für die Felder definiert werden oder die erwähnten Editor Bibliotheken referenziert werden. Je nach Typ der von Bibliothek müssen bestimmte

Konfigurationsdaten gesetzt und bestimmte Idiome in der JavaScript Datei eingehalten werden. Dies wird in dem Abschnitt 3.2 *H5P Dateiformat* genauer betrachtet.

Kapitel 3

H5P Library Entwicklung

In diesem Kapitel schauen wir uns an, wie wir H5P-Bibliotheken entwickeln und was zu beachten ist. Wir werden uns mit dem Einrichten der Entwicklungsumgebung, den Spezifikationen und Anforderungen für H5P-Bibliotheken sowie die Nutzung anderer H5P-Bibliotheken als Abhängigkeiten nutzen beschäftigen. Viele der in diesem Kapitel vorgestellten Anforderungen und Spezifikationen sind, wenn nicht anders angegeben, aus eigener Erfahrung mit dem Umgang mit H5P-Bibliotheken und der H5P-CLI entnommen.

3.1 H5P Entwicklungsumgebung

Die Entwicklung von H5P-Bibliotheken ist nicht trivial und erfordert eine entsprechende Entwicklungsumgebung, die eine H5P-Integration hat. In der Dokumentation von H5P wird empfohlen, die H5P-CLI zu verwenden, um eine solche Umgebung einzurichten. Die H5P-CLI ist, wie der Name suggeriert, ein Kommandozeilenwerkzeug, das nützliche Funktionen bietet und eine automatische Installation und Einrichtung von H5P-Core und Editor ermöglicht. Dies hat den sehr großen Vorteil, dass wir ohne großen Overhead eine Repräsentation einer Plattform mit H5P Plugin haben. Die Alternative wäre entweder ein CMS/LMS lokal mit dem H5P-Plugin zu hosten oder einen Docker-Container mit einem Image von eines CMS/LMS zu verwenden. Abgesehen vom Aufwand des Aufsetzens und der Konfiguration bietet die H5P-CLI gegenüber der selbst gehosteten Alternative Komfortfunktionen, wie etwa das automatische Neuladen von Content-Type-Instanzen im Webbrowser beim Speichern während der Entwicklung. Die H5P-CLI umfasst Funktionen, um mit Repositories von H5P-Bibliotheken zu interagieren. Wenn wir beispielsweise eine bestehende

```
h5p setup <library|repoUrl> [version] [download]
```

Listing 3.1: H5P Setup-Kommand

```
h5p utils pack [-r] <library> [<library2>...] [my.h5p]
```

Listing 3.2: H5P Utils-Pack-Kommand

H5P-Bibliothek als Abhängigkeit nutzen wollen, können wir diese mit der H5P-CLI in unsere Entwicklungsumgebung einbinden. Dabei kümmert sich die CLI auch darum, alle Abhängigkeiten der einzubindenden H5P-Bibliothek einzubinden. Dies passiert transitiv für alle Abhängigkeiten. Dafür nutzen wir den Listing 3.1-Befehl. Die genaue Dokumentation für diesen Befehl ist über den help-Befehl der H5P-CLI oder im Github Repository zu finden. [H5P25a] Nun könnten wir unter Angabe des Repository Links wie z.B. 'git@github.com:h5p/h5p-accordion.git' die Bibliothek und alle Abhängigkeiten mit der richtigen Version in unsere Entwicklungsumgebung einbinden. Wenn das Repository der Bibliothek unter der Organisation 'https://github.com/h5p' liegt, reicht es, den Namen des Repositories anzugeben. In dem Fall 'h5p-accordion'.

Darüber hinaus bietet die H5P-CLI einige weitere 'utils' Befehle, mit denen sich H5P-Bibliotheken validieren lassen und die als Wrapper für Git-Befehle dienen. Ein weiterer erwähnenswerter Befehl ist der Listing 3.2-Befehl, mit dem können wir unsere entwickelte H5P-Bibliothek in ein .h5p Dateiformat packen mit allen Abhängigkeiten. Die Dokumentation der Argumente und Flags ist nur über den 'help'-Befehl der H5P-CLI zu finden. Das Dateiformat ist im Grunde eine ZIP-Datei mit eigener Dateieindung zur Unterscheidung sowie einigen Konfigurations- und Metadateien. [H5P25h] Das ermöglicht uns, unsere entwickelte H5P-Bibliothek zwischen CMS/LMS zu transferieren, in denen das H5P-Plugin installiert ist.

3.1.1 H5P-CLI fehlende Dokumentation und Probleme

Die Dokumentation für die H5P-CLI ist jedoch entweder nicht mehr aktuell und unvollständig oder die Antworten aus den Befehlen sind teilweise vage oder nichtssagend. Beim Nutzen von Befehlen ist nicht klar, was genau validiert wird und Fehlermeldungen sind generisch. Die Dokumentation für die CLI Befehle auf der H5P-Webseite ist nicht mehr aktuell und teilweise gibt es Befehle die dort aufgeführt werden, die es nicht mehr gibt. So wird beispielsweise der Befehl 'h5p get <library>' aufgeführt der nicht mehr existiert. [H5P25c] Dies erschwert es neuen Entwicklern, die versuchen

sich in H5P einzuarbeiten, und führt zu Verwirrung. Ein weiteres Beispiel ist der Befehl „h5p utils pack“, der unsere H5P-Bibliothek unteranderm validiert. Allerdings ist unklar, was genau validiert wird. Beim Fehlschlagen erhalten wir die generische Fehlermeldung „no valid libraries found; use '-f' to skip validation“. Die einzige Möglichkeit die bestand war in den Source-Code zu schauen und zu versuchen herauszufinden was genau validiert wird. Nach einigen Versuchen zu verstehen was genau passiert, fällt auf, dass der Befehl Listing 3.3 die Existenz von `/.git/config` im Root-Verzeichnis der H5P-Bibliothek prüft. Warum dies für die Validierung wichtig ist, bleibt unklar. Der entwickelte Content-Typ war kein eigenständiges Git-Repository, sondern war als Unterverzeichnis in einem Git-Repository eingebunden. Das führte zum Fehlschlagen des Befehls. Jedoch ist das als Entwickler nicht ersichtlich, ohne in den Source-Code zu schauen, was auch keine Anforderung sein sollte. Wenn wir versuchen, die Validierung mit dem `'-f'` Flag zu umgehen, wird auch das Packen der Abhängigkeiten in die `.h5p` Datei übersprungen. Dieses Verhalten ist ebenfalls nicht dokumentiert, was ein Problem für Entwickler darstellt, die sich nicht mit der H5P-CLI auskennen.

3.1.2 Representation von anderen Plattformen

Was auch anzumerken ist, dass Bibliotheken, die in der H5P-CLI entwickelt wurden, nicht garantiert auf anderen Implementierungen von H5P funktionieren, da es entweder an fehlende Abhängigkeiten, technisch nicht kompatibelem Code oder anderen Implementierungsdetails liegen kann. Auch die von uns entwickelten Content-Typen sollten immer auf der Zielplattform getestet werden. Wo genau die Probleme liegen und welche Gründe dafür verantwortlich sind, wurde im Rahmen dieser Arbeit nicht weiter untersucht. Erfahrungsberichte anderer H5P-Entwickler, die auf h5p.org in Form von Blogeinträgen veröffentlicht wurden, sowie eigene Erfahrungen beim Versuch, H5P-Bibliotheken in Lumi, einem Editor zur Erstellung von Instanzen von Content-Typen, zu importieren, deuten jedoch darauf hin. An dieser Stelle kann der Erfahrungsbericht von Sviatoslav Tugeev von der Technische Hochschule Ingolstadt angeführt werden. [Tug24, 4.2.3 Limitations] Angesichts der Trittbretter in Form von fehlender Dokumentation und unklaren Fehlermeldungen, ist dies nicht verwunderlich.

3.2 H5P Dateiformat

H5P-Bibliotheken sind Sammlungen von Dateien mit teilweise spezifischen Dateinamen oder Idiotemen, die Voraussetzung sind, damit sie korrekt geladen werden können. Eine typische Ordnerstruktur


```
function checkRepository(dir, next) {
  fs.stat(dir + '/.git/config', function (error, stats) {
    if (error) return next(null);
    next(dir);
  });
}
```

Listing 3.3: Check Funktion in der Code Base von h5p-cli

für eine H5P-Bibliothek ist in Abbildung 3.1 dargestellt. Wir erkennen, dass der Name des Root-Ordners eine bestimmte Struktur hat. Dieser ist für das richtige Laden der H5P-Bibliothek relevant. Der genaue Zusammenhang wird an späterer Stelle in diesem Abschnitt genauer betrachtet.

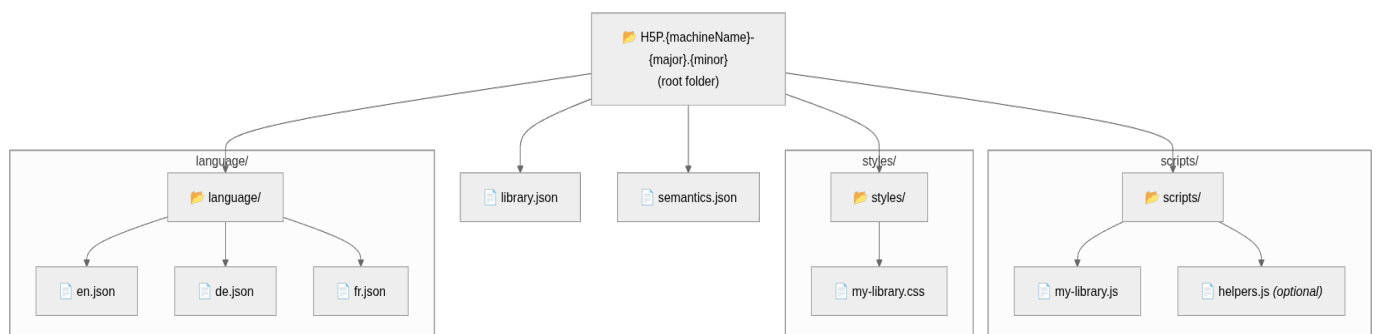


Abbildung 3.1: Typische Ordnerstruktur einer H5P-Bibliothek

Lokalisierungen von H5P-Bibliotheken werden unter dem Ordner 'language' abgelegt, hiermit werden wir uns nicht weiter beschäftigen.

Je nach Typ der H5P-Bibliothek sind verschiedene Dateien erforderlich. Wie bereits erwähnt, sind 'library.json' und eine JavaScript-Datei für alle Typen von H5P-Bibliotheken erforderlich. Wenn wir einen Content-Typ entwickeln, benötigen wir zudem eine 'semantics.json' Datei, für Daten, die im Autoren Interface angezeigt werden sollen.

Im Folgenden gehen wir auf die einzelnen Konfigurationsdateien ein. Wir beschreiben deren Inhalt und was zu beachten ist. Anschließend behandeln wir die JavaScript-Datei und die zu beachtenden Idiome und Konventionen.

Feld	Beschreibung
title	Anzeige Name der Bibliothek
machineName	Name zur Referenzierung
majorVersion	Major Versionsnummer
minorVersion	Minor Versionsnummer
patchVersion	Patch Versionsnummer
runnable	'0' oder '1' beschreibt ob Bibliothek alleine Ausführbar ist

Tabelle 3.1: Erforderliche Felder in der 'library.json' Datei

3.2.1 Library.json

Die 'library.json' ist eine Konfigurationsdatei, die Metadaten über die H5P-Bibliothek enthält. Es handelt sich um eine herkömmliche JSON-Datei, die eine Reihe von Feldern enthält, die für H5P relevant sind. Anhand dieser Informationen kann H5P die Bibliothek identifizieren und weiß wie, sie behandelt werden soll.

Erforderliche Felder sind in der Tabelle 3.1 aufgelistet. Nähere Informationen zu weiteren Feldern sind in der H5P-CLI Dokumentation zu finden. [H5P25d] Wichtig zu erwähnen ist, dass der 'machineName' identisch mit dem gleichnamigen Teil des Ordernamens aus der Abbildung 3.1 ist. Außerdem muss die ES5 Klasse, die in der JavaScript-Datei definiert ist, den gleichen Namen wie der 'machineName' haben. Zu den Format der JavaScript-Datei und den Idiomen kommen wir später im Abschnitt 3.2.3.

Je nachdem welchen Typ von H5P-Bibliothek wir entwickeln, muss 'runnable' entweder auf '0' oder '1' gesetzt werden. Anhand diesem Wert kann H5P erkennen, ob es sich um einen Content-Typ handelt oder nicht und zeigt nur Content-Typen als Option für Autoren an. Für Content-Typen wird 'runnable' auf '1' gesetzt, da diese als einzige eigenständig sich im bereitgestellten Bereich der Webseite darstellen können. Für Editor-Bibliotheken und API-Bibliotheken wird 'runnable' auf '0' gesetzt, da diese Bibliotheken nur in Kombination mit Content-Typen verwendet werden.

Zwei weitere Felder die relevant sind für fast alle H5P-Bibliotheken sind 'preloadedJs' und 'preloadedCss'. 'preloadedJs' sollte bei jeder H5P-Bibliothek gesetzt sein. Ohne das Setzen von 'preloadedJs' wird keine JavaScript-Datei geladen. Dies ist für keine H5P-Bibliothek, sinnvoll, da die JavaScript-Datei die Logik enthält und je nach Typ von Bibliothek für das Rendern des Inhalts der Bibliothek zuständig ist.

'preloadedCss' ist, wie der Name impliziert, der Pfad zur CSS-Datei der H5P-Bibliothek, welche von H5P in den Container der Bibliothekinstanz als stylesheet geladen wird. Zu beachten ist hier, dass H5P standardmäßige Styles für bestimmte HTML-Elemente hat, je nach Plattform.

Es können mehrere Dateien für 'preloadedJs' und 'preloadedCss' angegeben werden. Dabei wird einfach ein weiteres Key-Value Paar aus 'path' und dem relativen Pfad zur Datei angegeben.

Die Instanz der H5P-Bibliothek referenziert sowohl die definierten JavaScript-Dateien als auch die CSS-Dateien, im '<head>' -Tag des iframes, in dem die Instanz gerendert wird.

Ein weiteres erwähnenswertes Feld, das im Abschnitt 3.3 genauer betrachtet wird, ist 'preloaded-Dependencies'. Hier geben wir andere H5P-Bibliotheken an, die wir als Abhängigkeit für unsere H5P-Bibliothek benötigen. H5P bindet diese Bibliotheken im bereitgestellten Container im Head-Tag des iframes über script-Tags ein.

3.2.2 Semantics.json

Die 'semantics.json' ist eine Konfigurationsdatei, die die Struktur der im Autoren-Interface anzuzeigenden Daten beschreibt. Entwickler von Content-Typen definieren darin die erforderlichen Daten und ihre Struktur und H5P generiert daraus das Autoren Interface.

Dabei wird die Struktur definiert mit Hilfe von sogenannten 'Sementic Types', welche JSON Objekte sind. Die JSON Datei besteht aus einem Array dieser Objekte. Es gibt eine Vielzahl von 'Sementic Types', welche wiederum bestimmte Attribute unterstützen. Eine Übersicht ist in der Spezifikation zur 'semantics.json' zu finden. [H5P25f]

Interessant zu erwähnen ist, dass alle 'Sementic Types' die einen Input vom Autor darstellen, das 'widget'-Attribut haben. Dieses Attribut beschreibt, welches Widget im Autoren Interface für das Feld verwendet wird. Das 'widget'-Attribut kann entweder ein Standard-Widget sein, wie z.B. 'text', 'boolean', 'select' oder ein 'custom widget'. 'custom widgets' sind wiederum auch H5P-Bibliotheken.

Die Daten werden dabei entweder zur Konfiguration des Verhaltens des Content-Typs oder als dessen eigentlichen Inhalt des Content-Typs verwendet.

3.2.3 H5P-spezifische Vorgaben und Limitierungen für JavaScript

Damit H5P die JavaScript-Dateien korrekt laden kann, müssen wir bestimmte Idiome einhalten. Je nach Typ von H5P-Bibliothek gibt es unterschiedliche Vorgaben.

Bei der Entwicklung von H5P-Bibliotheken müssen wir außerdem berücksichtigen, dass der Code in verschiedenen Browsern und Versionen ausgeführt wird. Auch wenn ES6 Features von der überwiegenden Mehrheit der genutzten Browser und Browser Versionen unterstützt wird [use25], sind wir durch H5P limitiert welche Idiome wir nutzen können. Einschränkungen bestehen auch bei den neueren Features von ECMAScript, die noch nicht überwiegend unterstützt werden.

Wie wir trotzdem moderne JavaScript Features nutzen können, wird im Kapitel 4 genauer betrachtet.

In dem 'Hello World' Guide auf der H5P-Website werden wir in eine 'empfohlene' Code Struktur eingeführt. [H5P25e, Step 3] Hierbei handelt es sich tatsächlich um ein Format, welches wir einhalten sollten, damit H5P unsere Bibliothek korrekt laden kann und sie im Browser korrekt ausgeführt wird. Wir erkennen, dass hier ES5-kompatible Klassenstrukturen (d. h. Konstruktorfunktionen mit Prototyp-Vererbung) verwendet werden. Das Objekt H5P wird in den globalen Namespace, also in das window-Objekt des aktuellen Browserkontexts, geladen und fungiert dort als Namespace, über den H5P die übrigen Bibliotheksfunktionen referenzieren kann. An dieses Objekt hängen wir unsere Klasse an, die den gleichen Namen wie der 'machineName' in der 'library.json' haben muss. Die Klasse muss in einem IIFE (Immediately Invoked Function Expression) gekapselt sein. [Doc25b]

Die genauen technischen Spezifikationen von H5P und die Gründe für die Nutzung bestimmter Idiome sind nicht dokumentiert. Teilweise können wir bestimmte Idiome weglassen und H5P kann unsere Bibliothek trotzdem laden. Auch wenn H5P Open Source ist, ist das nachvollziehen anhand vom Code nicht trivial. Jedoch können wir Anhand von den Eigenschaften der genutzten Idiome Annahmen treffen. In den Coding Guidelines wird [H5P25b] angesprochen, den globalen Scope nicht zu verschmutzen. Da potenziell mehrere H5P-Bibliotheken gleichzeitig auf derselben Seite eingebettet sein können, wird versucht Kollisionen zwischen den Bibliotheken zu vermeiden. Beispielsweise bei gleichnamigen Funktionen oder Variablen. IIFE kapseln den Code in einem eigenen Scope ein, um Kollisionen zu vermeiden.

Außerdem können wir mithilfe der IIFE Notation Instanzen von jQuery übergeben und diese so eindeutig referenzieren. Das ist hilfreich, wenn im CMS/LMS bereits eine Instanz von jQuery

geladen ist. Bei Nutzung von anderen H5P-Bibliotheken als Abhängigkeit, müssen diese auch in die IIFE übergeben werden. Dazu mehr im nächsten Abschnitt 3.3.

Zu dem müssen wir eine Konstruktorfunktion definieren, die am Ende der Klasse zurückgegeben wird.

Für Content-Typen müssen wir unseren Konstruktor so definieren, dass er zwei Parameter annimmt. Der erste Parameter enthält die Daten, die im Autoren Interface eingegeben wurden. Der zweite Parameter ist ein Identifikator für die Instanz der H5P-Bibliothek, die geladen wird. Diese Parameter werden von H5P an den Konstruktor übergeben. Im Konstruktor setzen wir die ID der Instanz und entweder speichern die Daten direkt oder überschreiben nicht angegebene Daten mit Standardwerten. Dabei können wir, wie im 'Hello World'-Beispiel, jQuery nutzen oder auch Object.assign aus ES6 verwenden. Anzumerken ist: In der 'semantics.json' Datei können wir bereits Default-Werte für die Daten definieren und uns dies hier unter Umständen ersparen. Für Content-Typen müssen wir zudem eine Methode mit dem Namen 'attach' definieren und an das Prototyp der Klasse anhängen. Diese Methode wird von H5P aufgerufen und übergibt uns den bereitgestellten Container als ersten Parameter, mit der sich die Bibliothek um das Rendern des Inhalts kümmert.

Für API-Bibliotheken gibt es keine Vorgaben von H5P, wie der Konstruktor aussehen muss. Da API-Bibliotheken im Zusammenhang mit anderen Bibliotheken genutzt werden, hängt ihre Implementierung von den Anforderungen der jeweiligen API-Bibliothek ab.

Für Editor-Bibliotheken muss mindestens eine 'appendTo' Methode definieren werden, die sich ähnlich wie die 'attach' Methode für Content-Typen verhält. Jedoch sind Anforderungen kaum klar definiert und im Kontext der Arbeit auch nicht weiter untersucht worden. Es ist empfehlenswert, sich an bereits existierenden Editor-Bibliotheken zu orientieren.

Je nach Komplexität der zu entwickelnden Bibliothek, muss der Code in verschiedene Module und Dateien aufgeteilt werden. Dies erleichtert die Wartung und das Nachvollziehen von Code, denn wir können so Code in kohärente Einheiten aufteilen, die bestimmte Logik kapseln. Um dies zu ermöglichen, gibt es in ES6 Module Imports, [Doc25a] jedoch muss dafür im Script-Tag das Attribut 'type' auf 'module' gesetzt werden. [Doc25c] H5P tut dies nicht, sondern lädt alle angegebenen JavaScript-Dateien ohne das 'type' Attribut zu setzen.

Wollen wir unseren Code logisch kohärenten 'modularisieren', ohne auf einen Buildprozess der Code bundelt und transpiliert zurückzugreifen, muss jede Datei wieder in einem IIFE gekapselt sein

```
(function (MyLibrary){
  MyLibrary.Helpers = function(param1, param2) {
    // Logik der Helper Klasse
  };
})(H5P.MyLibrary);
```

Listing 3.4: H5P Helper Klasse in IIFE Kapselung

```
H5P = H5P || {};
```

```
H5P.MyLibrary = (function (){
  function MyLibrary(options, id) {
    // Konstruktor der H5P Bibliothek
  }

  MyLibrary.prototype.attach = function (container) {
    // Logik zum Rendern des Inhalts
  };

  // Weitere Methoden und Logik für die H5P Bibliothek

  return MyLibrary;
})();
```

Listing 3.5: H5P Klasse in IIFE Kapselung

und Helferfunktionen oder Klassen an das Prototyp der Hauptklasse angehängt werden, welches den gleichen Namen wie der 'machineName' in der 'library.json' hat.

Nehmen wir als Beispiel die, Struktur in der Abbildung 3.1. Dann haben wir eine zweite JavaScript-Datei 'helpers.js', die wir in der library.json unter 'preloadedJs' referenzieren. Nehmen wir an der 'machineName' ist 'MyLibrary', dann würde der Code wie in Listing 3.4 aussehen. Die gesamte Logik sollte in einem IIFE gekapselt sein und wir übergeben mit der Hilfe des globalen Objekts 'H5P' die Instanz der Bibliothek, die wir erweitern wollen. Wir können nun in unserer Hauptklasse 'MyLibrary' auf die 'Helpers'-Klasse zugreifen und diese nutzen. Diese Struktur sorgt wahrscheinlich für eine kollisionssichere Kapselung und laden der Bibliothek im Browserkontext. Ein konkretes Beispiel die H5P-Bibliothek 'H5P.Blanks'. [AS25a] Diese Struktur wird auch Revealing Module Pattern genannt.

Es sollte sich an die vorgestellten Strukturen gehalten werden bei der Bibliotheksentwicklung, sonst

kommt es zu unerwartetem Verhalten welcher nicht nachvollzogen werden kann. Ob dies an der H5P-Implementierung als Plugin und seinen damit einhergehenden Einschränkungen oder an der Bibliothek selbst liegt, ist für mich nicht klar. Zu vermuten ist, dass H5P in einer Zeit entwickelt wurde, in der JavaScript einen ganz anderen Standard hatte, und die Vorgaben aus dieser Zeit stammen. Selbst nach der Einführung von ES6 im Jahr 2015 fanden die ES6 Spezifikationen erst in 2018 übergreifenden Unterstützung. Daraus resultierend mussten wahrscheinlich viele der Vorgaben beibehalten werden, um die Abwärtskompatibilität zu gewährleisten. Durch rückwärtskompatible JavaScript-Idiome bleibt H5P unabhängig von modernen Build-Tools, auf älteren sowie künftigen Browser-Versionen lauffähig und vereinfacht wahrscheinlich die Wartung für die Maintainer. Diese technologischen Entscheidungen haben höchstwahrscheinlich die langfristige Beständigkeit und den Erfolg des Projekts gefördert.

3.3 Nutzung von anderen H5P-Bibliotheken als Abhängigkeit

Wie bei jedem Softwareprojekt ist es erstrebenswert, auf bereits bewährte Software zurückzugreifen und auf ihr aufzubauen. Zum einen beschleunigt dies die Entwicklung, da wir auf bestehende Lösungen für Anforderungen zurückgreifen und zum anderen werden Bugs vermieden, die in weitverbreiteten Bibliotheken bereits behoben wurden. H5P bietet eine Vielzahl von Open Source Bibliotheken aller Typen an, die wir als Abhängigkeit in unsere H5P-Bibliothek einbinden können und so auf deren Funktionalität aufbauen können.

Beispielsweise können wir auf 'H5P.Question' zurückgreifen für eine einförmige Lösung für Frage-Typen; Content-Typen die interaktive Frage für den Nutzer darstellen. [AS25b] Sie liefert eine Basis-Klasse mit Logik und Auswertung für Frage-Typen, um das Erstellen solcher Bibliotheken zu vereinfachen. Die meisten Content-Typen nutzen diese Bibliothek als Abhängigkeit, um die Logik für Fragen zu kapseln und sie somit nicht selbst implementieren zu müssen.

Um andere H5P-Bibliotheken als Abhängigkeit zu nutzen, müssen wir einige Dinge beachten. Zunächst sollten wir sicherstellen, dass wir die H5P-Bibliothek in unserer Entwicklungsumgebung eingebunden haben. Dafür nutzen wir den Listing 3.1-Befehl der H5P-CLI.

Jetzt können wir in der 'library.json' Datei, die als Abhängigkeit gewünschte H5P-Bibliothek, unter 'preloadedDependencies' angeben. Hierbei ist zu beachten, dass wir die Angaben unter 'preloadedDependencies' der genutzten H5P-Bibliothek, in unsere Bibliothek übernehmen. Dies

stellt sicher, dass wir die korrekten Versionen der Abhängigkeiten nutzen und H5P die Abhängigkeiten korrekt laden kann. H5P scheint transitive Abhängigkeiten, ohne das übernehmen, lösen zu können für das Laden in den Browserkontext, jedoch ist dies nicht klar dokumentiert. Bibliotheken, die beispielsweise 'H5P.Question' als Abhängigkeit haben, übernehmen ebenso die Abhängigkeiten von 'H5P.Question' in Ihre 'preloadedDependencies'.

Erweitern wir einen Content-Typ, wie 'H5P.Blanks', müssen wir auch die 'semantics.json' Datei übernehmen und mit unseren benötigten Daten erweitern, sonst wird das Autoren Interface ohne die Daten für die abhängige Bibliothek generiert.

Außerdem müssen wir in unserer JavaScript-Datei die abhängige Bibliothek in der IIFE übergeben, damit wir auf die abhängige Klasse zugreifen können. Ähnlich wie bei der Nutzung von mehreren JavaScript-Dateien, in dem Listing 3.4 dargestellt.

Wie wir mit dem nun verfügbaren Code der Abhängigkeit umgehen, hängt von der Art der H5P-Bibliothek und ihrem Verwendungszweck ab.

API-Bibliotheken können eine Grundklasse bereitstellen die wir Erweitern um einen Typ von dieser API-Bibliothek zu erstellen. Die Nutzung der API-Bibliothek sollte im besten Fall vom Ersteller dokumentiert sein.

Um die Funktionalität eines vorhandenen Content-Typs sauber zu erweitern, nutzt man den Content-Typ als Basisklasse und ruft im Konstruktor der abgeleiteten Klasse explizit den Basisklassen-Konstruktor auf. Hierfür werden ES5 Idiome verwendet.

Schauen wir uns dazu Listing 3.6 an. Hier wird die Basisklasse 'H5P.BaseLibrary' als Parameter an die IIFE übergeben und die abgeleitete Klasse 'H5P.MyLibrary' wird definiert. Im Konstruktor der abgeleiteten Klasse wird der Basisklassen-Konstruktor mit der Referenz auf das aktuelle Objekt (this) und den erwarteten Parametern aufgerufen. Auf diese Weise wird das aktuelle Objekt (this) mit sämtlichen Initialisierungsparametern an die Ursprungsklasse durchgereicht, sodass deren etablierte Logik samt Zustandskonfiguration unverändert weitergenutzt werden kann. Außerdem sehen wir in der vorletzten Zeile, dass der Prototyp der abgeleiteten Klasse auf den Prototyp der Basisklasse gesetzt wird. Dies ermöglicht es der abgeleiteten Klasse, auf alle Methoden und Eigenschaften der Basisklasse zuzugreifen. In der letzten Zeile wird der Konstruktor der abgeleiteten Klasse auf 'MyLibrary' gesetzt, da ansonsten der Konstruktor der Basisklasse referenziert wird wegen der Herstellung der Prototyp-Vererbung. Dies scheint für H5P-Core notwendig zu sein, da es bei der


```
H5P.MyLibrary = (function (BaseLibrary) {  
  function MyLibrary(params, id) {  
    // Logik der Sub Klasse  
  
    // Aufruf des Basisklassen-Konstruktors  
    BaseLibrary.call(this, params, id);  
  };  
  
  // Prototyp-Vererbung  
  MyLibrary.prototype = Object.create(BaseLibrary.prototype);  
  // Setzen der richtigen Konstruktor-Referenz  
  MyLibrary.prototype.constructor = MyLibrary;  
  
  return MyLibrary;  
})(H5P.BaseLibrary);
```

Listing 3.6: H5P Prototyp Erweiterung in IIFE Kapselung

Instanziierung der H5P-Bibliothek auf den Konstruktor zugreift. Dies ist möglicherweise auch in anderen Szenarien der Fall, was jedoch nicht eindeutig aus der Dokumentation ersichtlich ist.

Nun können wir in der abgeleiteten Klasse zusätzliche Methoden und Eigenschaften definieren, oder bestehende Methoden überschreiben, um das Verhalten des Content-Typs zu erweitern oder anzupassen. Dafür ist ein gutes Verständnis der Basisklasse erforderlich.

Kapitel 4

Nutzung von Package Managern

Ähnlich wie bei der Nutzung von H5P-Bibliotheken als Abhängigkeiten, ist es auch erstrebenswert, auf JavaScript-Bibliotheken von Package Managern wie 'npm' zurückzugreifen, und zwar aus ähnlichen Gründen. JavaScript-Bibliotheken nutzen entweder ES6 Module oder CommonJS Module, um Code zu kapseln und zu organisieren. H5P bietet jedoch keine Möglichkeit, ES6 Module direkt zu nutzen, da wie schon erwähnt, H5P JavaScript-Dateien ohne das 'type' Attribut lädt. CommonJS ist ein Module-System, das nur in Node.js und nicht im Browser unterstützt wird.

Auch wenn wir es irgendwie ermöglichen würden, dass H5P JavaScript-Dateien als Module lädt und somit alle Abhängigkeiten aufgelöst werden, hätten wir, je nach Umfang der genutzten JavaScript-Bibliotheken, ein umfangreiches Paket mit mehr Daten als notwendig, die beim Laden vom Server an den Client übertragen werden müssen. Dadurch würde sich der Speicherbedarf vergrößern und die Ladezeit verlängern, was zu einer spürbar negativen Performanz führen könnte. Gerade ungenutzter Code erhöht die Gesamtgröße zusätzlich.

Um den Abhängigkeitsgraphen aufzulösen und den Code zu minimieren, benötigen wir einen Buildprozess, der die Abhängigkeiten auflöst und die ES6 Module oder CommonJS Module in ein Format transpilieren, das H5P versteht und laden kann. In diesem Prozess können wir auch dafür sorgen, dass moderneres JavaScript (ES5+) in ein abwärtskompatibles Format transpiliert wird, das in älteren Browsern funktioniert (ES5). Entwickler können so moderne JavaScript Features nutzen, ohne sich um die Abwärtskompatibilität oder Limitierungen von H5P kümmern zu müssen.

Die in diesem Kapitel beschriebenen Lösungen, um Abhängigkeiten aufzulösen, den Code zu optimieren und zu transpilieren, wurden maßgeblich erarbeitet durch das Entwickeln von den H5P

Content-Typen in Kapitel 5. Da es keine Dokumentation für einen solchen Buildprozess gibt, wurde sich an Open Source H5P-Bibliotheken orientiert, die bereits Webpack und Babel nutzen. Es wurde versucht, die Konzepte und Ideen zu abstrahieren und so eine generische Lösung zu finden, die für die meisten H5P-Projekte anwendbar ist.

4.1 Webpack und Babel

Ein Tool, das wir für den Buildprozess nutzen können, ist Webpack. Webpack ist ein Modul-Bundler, der einige zusätzliche Features bietet und unseren Code zusätzlich optimiert sowie ungenutzten Code entfernt mit einem Verfahren der sich 'Tree Shaking' nennt. [Web25e]

Webpack bekommt als Input eine oder mehrere JavaScript-Dateien als Entry-Punkte und erstellt daraus intern einen Abhängigkeitsgraphen, der alle Abhängigkeiten auflöst und in eine oder mehrere Ausgabedateien bündelt. Je nach gesetzter Environment-Variable kann Webpack den Code für die Entwicklung oder für die Produktion optimieren. Wenn wir den Code für die Entwicklung generieren, werden Metadaten wie Debugging-Informationen und Source Maps generiert und oder behalten, die für Browser-Entwicklungstools nützlich sind. Entwickler können so den Code einfacher debuggen und verstehen im Browser. Generieren wir den Code der als Asset an den Client ausgeliefert werden soll, also für Produktion, minimiert und optimiert Webpack den Code, um die Bundlegröße zu verringern und effektiv das Laden zu verkürzen. Ungenutzter Code aus den Abhängigkeiten wird erkannt und entfernt, sodass nur der tatsächlich genutzte Code im Bundle enthalten ist. [Web25f; Web25e]

Im Buildprozess von Webpack können wir auch 'Loaders' nutzen, um verschiedene Dateitypen zu verarbeiten oder zu transformieren. [Web25f, Loaders] Dies ermöglicht es uns, Assets wie CSS zu bündeln oder unseren JavaScript Code zu transpilieren. Um mit der Limitierung von H5P umzugehen, dass wir auf ES5 Idiome beschränkt sind, können wir den Babel JavaScript-Compiler nutzen. [Bab25d]

Babel liegt als 'Loader' für Webpack vor und transformiert dabei, je nach Konfiguration, den Code von ES5+ zu ES5. Babel kommt mit einer eigenen Konfigurationsdatei, in der wir 'presets' nutzen können, die eine Vielzahl von Übersetzungen von Idiomen bereitstellen. [Bab25b] Wir können auch 'plugins' definieren, die uns bestimmte Übersetzungen für bestimmte Idiome bereitstellen.

Wollen wir beispielsweise ES6 Klassen nutzen und in H5P erwartete Idiome transpilieren, können wir das Babel Plugin 'plugin-transform-classes' nutzen. [Bab25c] In den Beispielen der Dokumentation von diesem Plugin, sehen wir wie das Output Format, das Babel generiert, genau den Anforderungen von H5P entspricht.

Wir können nun ES6 Klassen schreiben und diesen Vererbungs Idiome nutzen und Babel transpiliert den Code in ein Format, das H5P erwartet und laden kann.

Babel und Webpack sind sehr flexible Tools, die uns eine Menge an Konfigurationsmöglichkeiten bietet und je nach Anforderungen angepasst werden können.

4.2 Einrichten eines Buildprozesses für H5P

Da Webpack ein sehr allgemeines Tool ist für viele Anwendungsfälle, ist der Einstieg, wie wir ihn hier beschreiben, nicht der einzige Weg, um Webpack zu konfigurieren. Der hier beschriebene Weg ist ausreichend für die meisten H5P-Projekte und kann je nach Anforderungen angepasst und erweitert werden. Es werden H5P-spezifische Eigenheiten und Entscheidungen aufgeführt und sonst auf die jeweilige Dokumentation verwiesen.

Um Webpack und Babel in einem H5P-Projekt zu nutzen, müssen wir zunächst die benötigten Pakete installieren. Wir beziehen uns in diesem Abschnitt auf den 'npm' Package Manager, der in der Node.js-Umgebung weit verbreitet ist. Für ähnliche Package Manager sind die Schritte und Konzepte ähnlich.

Wollen wir bestimmte JavaScript-Bibliotheken nutzen, die wir über den Package Manager beziehen, müssen wir zunächst unser Projekt mit dem Package Manager initialisieren. Dafür nehmen wir an, wir befinden uns in einem H5P-Projektverzeichnis. Nun können wir unsere JavaScript-Bibliotheken über den Package Manager installieren. Diese können wir in unserem H5P Projekt über ES6 Module oder CommonJS Module importieren. Des Weiteren installieren wir Webpack und das Kommandozeilen-Interface von Webpack in unser Projekt. Eine Anleitung, wie wir Webpack installieren, finden wir in der Webpack-Dokumentation. [Web25c]

Wir werden Webpack über eine Konfigurationsdatei konfigurieren, die wir im Projektverzeichnis anlegen. [Web25b] Der Vorteil davon ist dass wir die Konfigurationdatei versionieren können und

somit effektiver teilen können statt manuell Befehle zuschreiben und auszuführen.

Wir definieren nun in der Konfigurationsdatei, welche Dateien wir als Entry-Punkte für Webpack nutzen wollen. Die Datei in der wir unseren Code schreiben, geben wir Webpack als Startpunkt. Webpack erstellt den Abhängigkeitsgraphen und bündelt alle mit einander abhängigen Dateien in einen JavaScript-Datei. In dieser generierten Datei befindet sich unser H5P Code und all der Code aus den JavaScript-Paketten, den wir referenzieren.

Wollen wir unseren Code transpilieren, müssen wir den Entsprechenden Loader in der Konfiguration definieren. [Web25d] Webpack kümmert sich dann um die Ausführung des Loaders und Parsed die Dateien durch den Loader, um den Code zu transformieren. Eine Liste von verfügbaren Loaders finden wir in der Webpack-Dokumentation.

Für Babel nutzen wir den 'babel-loader'. [Web25a] Den 'loader' definieren wir, wie in der Webpack-Dokumentation beschrieben, für alle Dateien, die mit '.js' enden. Wir brauchen eine weitere Konfigurationsdatei für Babel, die wir ähnlich wie die Webpack-Konfigurationsdatei im Projektverzeichnis anlegen. Es gibt einige Möglichkeiten für den Typ von Konfigurationsdatei, die in der Babel-Dokumentation beschrieben sind. [Bab25a] Wir nutzen die 'File-relative configuration'. Die Datei nennen wir '.babelrc' und definieren darin die Presets und Plugins, die wir nutzen wollen.

Eine typische Ordnerstruktur könnte dabei wie in Listing 4.1 aussehen. Wir nutzen dabei den Ordner 'src' für unseren Code der als Input dient und den wir versionieren. Der Ordner 'dist' ist für das Bundle, aus dem Buildprozess.

Ein Template für 'webpack.config.js' und '.babelrc', das wir für H5P nutzen können und je nach Anforderungen anpassen können, könnte wie in Listing 4.4 und Listing 4.5 aussehen. Hier nutzen wir die 'NODE_ENV' Environment-Variable, für das bestimmen des Buildprozesses, und den Package Namen aus der 'package.json' Datei, für den Namen der gebündelten JavaScript-Datei.

Wichtig ist, dass wir in unserer 'library.json' Datei den Pfad zur gebündelten JavaScript-Datei angeben. Wenn wir unsere Webpack-Konfiguration so angepasst haben, wie beschrieben, können wir den Pfad zu der gebündelten JavaScript-Datei in der 'library.json' Datei angeben, mit dem Namen unseres Packages. Der Pfad sollte relativ zum H5P-Projektverzeichnis sein, also in unserem Fall 'dist/npm_package_name.js'.

Wir können nun in unserer package.json Datei ein Skript definieren, das den Buildprozess ausführt.

```
h5p-project/  
- node_modules/  
- src/  
- dist/  
- package.json  
- package-lock.json  
- .babelrc  
- webpack.config.js  
- library.json  
- semantics.json
```

Listing 4.1: Ordnerstruktur für H5P mit Webpack und Babel

```
webpack --watch --stats-error-details
```

Listing 4.2: Watch Skript für das Development Environment

Zwei empfohlene Skripte sind 'build' (Listing 4.3) und 'watch' (Listing 4.2). Das 'build' Skript führt den Buildprozess einmal aus und erstellt das Bundle. Hier setzen wir die Environment-Variable 'NODE_ENV' auf 'production', um den Code für die Produktion zu optimieren und Metadaten zu entfernen. Dazu nutzen wir das Package 'cross-env' [npm25] für das setzen der Environment-Variable. Da das 'build' Skript deutlich langsamer ist und es manuell ausgeführt werden muss, nutzen wir ein weiteres Skript, das 'watch' Skript, um den Buildprozess im Hintergrund laufen zu lassen, wenn Änderungen am Code automatisch erkannt werden. Da hier die Environment-Variable 'NODE_ENV' nicht gesetzt wird, ist der Buildprozess viel schneller und besser für die Entwicklung geeignet. Außerdem liefert er uns mehr Debugging-Informationen für den Buildprozess durch das Flag '--stats-error-details'.

```
cross-env NODE_ENV='production' webpack
```

Listing 4.3: Build Skript für das Production Environment

```
const path = require('path');
// Enviroment für den Buildprozess
const nodeEnv = process.env.NODE_ENV || 'development';
// Name des Packages, die in der package.json definiert ist
const libraryName = process.env.npm_package_name;

module.exports = {
  mode: nodeEnv,
  // Kontext für die Auflösung der Pfade
  context: path.resolve(__dirname, 'src'),
  // Name der Entry-Punkt Datei für Webpack
  entry: './my-library.js',
  devtool: nodeEnv === 'production' ? false : 'inline-source-map',
  output: {
    filename: `${libraryName}.js`,
    path: path.resolve(__dirname, './dist')
  },
  module: {
    rules: [
      {
        // Babel Loader für JavaScript Dateien
        test: /\.js$/,
        exclude: /node_modules/,
        use: ['babel-loader']
      }
    ]
  },
  stats: {
    colors: true
  }
};
```

Listing 4.4: Template für die Webpack Konfiguration

```
{
  "presets": ["@babel/preset-env"],
  "plugins": [
    // Plugins für bestimmte Babel Transformationen
  ]
}
```

Listing 4.5: Template für die Babel Konfiguration

Kapitel 5

Entwickelte H5P-Bibliotheken

In diesem Kapitel werden die entwickelten H5P-Bibliotheken vorgestellt, die auf den in den vorherigen Kapiteln beschriebenen Konzepten basieren. Mein erster Versuch einen Lückentext zu implementieren und mit einem Code Highlighting Package zu kombinieren, ist 'H5P.SyntaxHighlighting'. Diese Bibliothek hat maßgeblich zum Verständnis von H5P und H5P-Bibliotheken beigetragen. Die Bibliothek sollte wie ein Prototyp betrachtet werden. 'H5P.SyntaxHighlighting' ist eine Content-Typ-Bibliothek, die es ermöglicht, Code-Snippets mit Lückentexten zu erstellen. Sie basiert auf keiner anderen H5P-Bibliothek, sämtlicher Code wurde selbst implementiert. Da es keine triviale Aufgabe ist, eine nutzerfreundliche Lösung zu entwickeln, die vielen Anforderungen gerecht wird, ist es im Rahmen dieser Arbeit erstrebenswert, auf einer bestehenden Lösung aufzubauen. Dies ist in den meisten anderen Softwareprojekten ebenfalls sinnvoll. Daraus ergab sich die zweite entwickelte H5P-Bibliothek, 'H5P.SyntaxBlanks', die auf 'H5P.Blanks' basiert. Die hierbei angeeigneten Konzepte sind ES5 Vererbungen und die Erweiterung von H5P-Bibliotheken, welche nicht in der offiziellen Dokumentation von H5P beschrieben sind. 'H5P.Blanks' bietet eine bewährte Lösung für Lückentexte mit einer Vielzahl an Konfigurationsmöglichkeiten und Features. Die Bibliothek ist eine Implementierung, die eine Subklasse von 'H5P.Question' ist und eine einheitliche Logik und Darstellung von Fragen bereitstellt. Die Herausforderung bestand darin, den Code von 'H5P.Blanks' zu verstehen und an der richtigen Stelle zu erweitern, um es den Autoren zu ermöglichen, den Lückentext als Code-Snippet darzustellen und den Code zu highlighten.

5.1 H5P.SyntaxHighlighting

Die Grundidee hinter der Bibliothek war es, den Autoren die Möglichkeit zu geben, Code-Snippets mit Lückentexten zu erstellen. In Abbildung 5.1 ist der entwickelte Prototyp zu sehen. Dafür kann der Code im Autoren-Interface in einem WYSIWYG-Editor eingegeben werden. Der Autor definiert die Lücken im Code, indem er Textstellen durch Platzhalter ersetzt. Die Platzhalter haben das Format `{{gapX}}`, wobei X die fortlaufende Folge von 1 bis N ist. Der Autor muss die ersetzten Textstellen nun in der richtigen Reihenfolge als Lösungen angeben. Dafür wird im Autoren-Interface eine Liste von Lösungen angezeigt, die der Autor in der numerischen Reihenfolge der Lücken eingeben muss. Wobei die erste Lösung für die Lücke `{{gap1}}` ist, die zweite Lösung für die Lücke `{{gap2}}` und so weiter.

Die Logik der Bibliothek rendert über die `attach`-Methode den Container für den Code-Snippet. Der Code wird in einem `pre`-Tag mit einem `code`-Tag gerendert. Bevor wir den Code in den Container einfügen, wird der Code mit der `highlight.js`-Bibliothek geparkt und formatiert. Die Rückgabe ist ein HTML-String, der den formatierten Code mit Container mit entsprechenden Klassen für das Syntax-Highlighting enthält. Außerdem erstellen wir einen `button`-Element, mit dem der Nutzer seine Lösungen für die Lücken auswerten kann. In einem `div`-Element mit der Klasse `'feedback'` wird das Feedback für den Nutzer angezeigt, nach dem klick auf den Button.

Die Methode `replacePlaceholdersWithInputs` wird aufgerufen auf den Inhalt des `code`-Tags angewendet. Sie stellt die eigentliche Logik bereit, um die Platzhalter durch Eingabefelder zu ersetzen. Sie nutzt den DOM-Baum und läuft durch jeden Text-Node des Subbaums des `code`-Tags. Findet sie einen Treffer mit dem regulären Ausdruck, wird der Text-Node durch ein `input`-Element ersetzt, das die Klasse `'gap-field'` hat und das Attribut `'data-gap-index'` mit der Nummer aus `'gapX'` hat, wobei X die Nummer der Lücke ist. Die ID bekommen wir durch den regulären Ausdruck `Match`, da wir ein Capture Group für die Nummer der Lücke definiert haben.



Abbildung 5.1: Entwickelter Prototyp 'H5P.SyntaxHighlighting'

5.2 H5P.Blanks

Die Idee für die Bibliothek ist es, auf einer bewährten Lösung für Lückentexte aufzubauen, die eine Vielzahl an Konfigurationsmöglichkeiten und Features bietet die ein Autor für Lückentexte benötigt. In Abbildung 5.2 ist die Bibliothek zu sehen. 'H5P.Blanks' ist eine Content-Typ-Bibliothek, die auf 'H5P.Question' basiert und genau diese Anforderungen erfüllt. Meine Herangehensweise war es, die Bibliothek zu erweitern, um den Autoren die Möglichkeit zu geben, den Text als Code-Snippet darzustellen mit Syntax-Highlighting. Dafür habe ich im Autoren-Interface eine Checkbox hinzugefügt, der die Bibliothek konfiguriert den Text als Code-Snippet darzustellen. Außerdem war es eine Herausforderung den Code von 'H5P.Blanks' in einem angemessenen Umfang zu verstehen, um die richtigen Stellen zu finden, an denen ich die Logik erweitern kann.

Der erste Versuch war es die Funktion `createQuestions` zu überschreiben und das Highlighting-Package auf die erstellten HTML-Strings anzuwenden. Das hat nicht funktioniert, da die beiden genutzten Packages, 'highlight.js' und 'prism.js', nicht mit `input`-Elementen umgehen können oder diese aus dem Text entfernen.

Der zweite Ansatz war es, die Funktion `handleBlanks` zu überschreiben und zu erweitern. Die Funktion wird in `createQuestions` aufgerufen, um die mit einem Asterisk markierten Textstellen zu ersetzen und die Lösungen zu erfassen.

Dies löste das Problem jedoch nicht vollständig, da die Highlighting-Packages das Asterisk als Teil des Codes mit hervorheben. Somit befanden sich in den Lösungen HTML-Tags, die vom Highlighting-Package stammten.

Es wurden zwei Ansätze ausprobiert, um das Problem zu lösen. Ein Ansatz war der Versuch, den markierenden Charakter durch ein Unicode-Zeichen zu ersetzen, das nicht von den Packages gestylt wird. Dieser Ansatz löste das Problem der Verschmutzung der Lösungen durch die Highlighting-Packages nicht, wenn in der markierten Lücke ein programmiersprachenspezifisches Zeichen vorkommt, dieses Zeichen von den Packages gestylt wird. Ein Beispiel hierfür ist das Wort 'function' in JavaScript. Das Überschreiben des markierenden Charakters erwies sich jedoch als nützliches Feature, welches in das Autoren-Interface eingebaut wurde. Der zweite Ansatz bestand darin, alle HTML-Tags durch einen leeren String zu ersetzen, bevor die Lösungen der Lücken gespeichert wurden. In der `handleBlanks`-Funktion wurden die Lösungen bereits, von Elementen aus dem WYSIWYG-Editor bereinigt. Durch das anpassen des regulären Ausdrucks, der die Elemente

matchet, um jede Art von HTML-Tags zu entfernen, konnte das Problem gelöst werden.

Es wurde ein weiteres Package 'prism.js' zum Syntax-Highlighting eingeführt [Pri25], das mehr Features bietet und ein Open-Source Plugin für Babel hat [DiG25], das es uns ermöglicht, verschiedene Features zu konfigurieren, wie beispielsweise Code Blöcke mit Zeilennummern darzustellen.

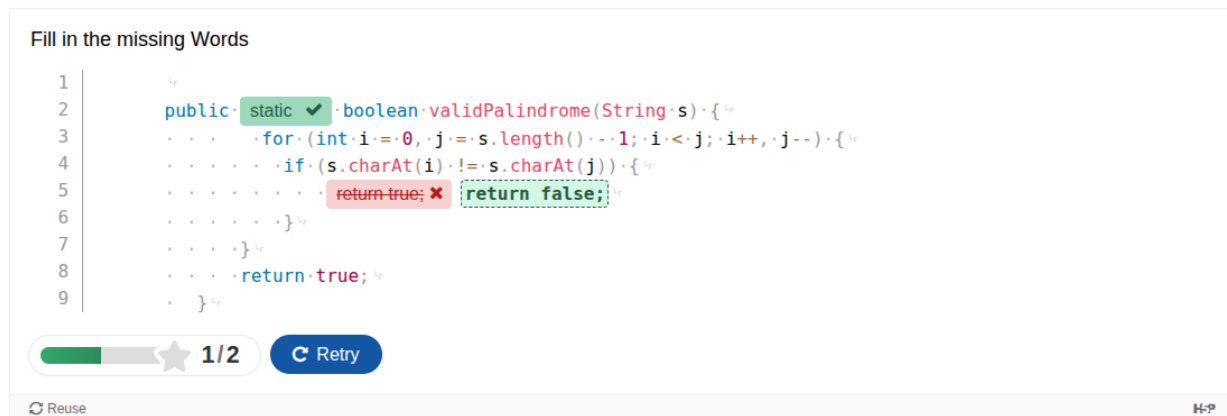


Abbildung 5.2: Entwickelter H5P-Fragetyp 'H5P.SyntaxBlanks'

Kapitel 6

Java Code Kompilierung und Interpretation in H5P

Im Rahmen dieser Arbeit wurde sich dem Konzept der Java Code Kompilierung und Interpretation im Browser in H5P gewidmet. Es wurden das grundlegende Konzept für das Entwickeln eines im Browser laufenden Java Compilers und Interpreters untersucht die in einer H5P-Bibliothek gewrappt werden können. Durch die Möglichkeit, Code direkt im Client auszuführen und gegebenenfalls auszuwerten, ergeben sich neue Möglichkeiten für interaktive Lerninhalte. Konkrete Anwendungsfälle könnten Code-Aufgaben mit Auswertung des Codes sein, ähnlich wie bei 'Codeforces' oder 'LeetCode'. Der entscheidende Vorteil durch das Ausführen im Client ist, dass kein Server mit dem einhergehenden technischen Overhead und Kosten benötigt wird. Einen solchen Server müssten wir vor böswilligem Code schützen, der unsere Anwendung missbrauchen könnte.

Wir bräuchten also ein Backend, welches im Browser läuft und das Äquivalent zu dem 'javac'-Befehl ist und uns somit Java-Bytecode generiert. Dazu bräuchten wir eine Implementierung der Java Virtual Machine im Browser, die den generierten Bytecode ausführen kann.

Um Java-Code auszuführen muss dieser zunächst in Java-Bytecode kompiliert werden. Es wäre ein hilfreiches Feature, wenn Nutzer ihren Code direkt im Browser kompilieren und ausführen könnten. So müssen sie ihren Code nicht selbst kompilieren und wir haben die Kontrolle über den Kompilierungsprozess. Das könnte hilfreich sein, wenn wir nur bestimmte Java-Features unterstützen wollen oder können. Dafür kommt GraalVM infrage, da es einen Java-Compiler bietet, der zu einem WASM-Modul ('javac.wasm') kompiliert werden kann und somit im Browser ausgeführt werden kann. [Goo25] Eine Demo dazu ist auf einer GraalVM-Webseite zu finden [Gra25]. Die

Schwierigkeit hierbei ist es H5P dazu zu bringen, den WASM-Code zu laden. Möglicherweise könnten wir einen Buildprozess nutzen, jedoch ist unklar, inwieweit dieser für WASM nützlich ist.

Es gibt einige Open-Source-Projekte, die versuchen Java-Code im Browser auszuführen. Ein vielversprechendes Projekt ist Doppio, eine in JavaScript geschriebene Java Virtual Machine. [JV14] Die Arbeit hat sich mit einer JavaScript basierten Runtime-Implementierung beschäftigt, die unter anderem DoppioJVM beinhaltet, sodass JVM-Programme ohne Anpassungen direkt im Browser ausgeführt werden können. Die schlechte Laufzeitperformance von DoppioJVM ist dabei je nach Anwendungsfall ein Hindernis, jedoch im Kontext eines Lehrenden der Interaktive Aufgaben erstellen möchte, wahrscheinlich zu vernachlässigen. Da die Arbeit aus dem Jahr 2014 stammt, ist Doppio nicht auf dem neuesten Stand einer JVM-Implementierung. Nach weiterer Evaluation könnte vielleicht eine rein WASM-basierte Lösung wie GraalVM für das Ausführen von Java-Bytecode im Browser eher infrage kommen.

Ein mögliches Konzept wäre, zwei API-Bibliotheken zu erstellen, die jeweils eine der beiden Lösungen umhüllen. Die APIs würden dann Funktionalitäten bereitstellen, die Entwickler in Content-Typen nutzen können. Zu beachten ist jedoch, dass die vorgestellten Lösungen Pakete mit mehreren Megabyte Größe sind, die an den Client gesendet werden müssen.

Im Rahmen dieser Arbeit konnten keine lauffähigen Prototypen oder MVPs als H5P-Wrapper für GraalVM oder Doppio implementiert werden. Die hier genannten Ansätze und Konzepte könnten in zukünftigen Arbeiten weiterverfolgt werden.

Kapitel 7

Fazit

7.1 Related Works

Eine Alternative zu H5P ist der in Moodle integrierte Fragentyp *Code Runner*. Er bietet die Möglichkeit, Code von Lernenden in verschiedenen Programmiersprachen zu eingeben und auszuwerten. Darüber hinaus kann er genutzt werden, um interaktive Inhalte zu erstellen. Es wurde sich jedoch für H5P entschieden, da es für mehrere Content-Management-Systeme verfügbar ist und eine größere Auswahl an bereits existierenden Fragentypen bietet.

7.2 Diskussion der Ergebnisse

In dieser Arbeit wurden Fragentypen erstellt, mit denen sich Lückentexte für Code-Snippets erstellen lassen. Damit konnte die ursprüngliche Problemstellung teilweise gelöst werden.

Während der Entwicklung sind wir auf sporadisch dokumentierte Spezifikationen und Konventionen von H5P gestoßen. Dies erschwerte die Entwicklung von H5P-Bibliotheken. Offizielle Dokumentationen waren teilweise unvollständig oder veraltet. Die einzige Form von Erklärungen für bestimmte Idiome waren Foren- und Blogbeiträge der Community. Teilweise musste der Quellcode analysiert werden, um bestimmtes Verhalten nachzuvollziehen. Dies war hinderlich für die eigentliche Motivation der Arbeit, da es zu unerwartetem Verhalten und Bugs führte, die nicht sofort nachvollziehbar waren.

H5P hat bestimmte Altlasten aus der Zeit vor den ES5+ Standards, die harte Rahmenbedingungen darstellen. Diese erzwingen Workarounds für Entwickler, die moderne JavaScript-Features oder -Bibliotheken nutzen möchten. Dadurch entsteht technischer Mehraufwand, der in der H5P-Community bislang nur unzureichend dokumentiert ist.

Es wurden Limitierungen und Herausforderungen bei der Entwicklung von H5P-Bibliotheken in dieser Arbeit aufgedeckt und für zukünftige Entwickler dokumentiert und diskutiert.

Es wurde sich mit modernen und Pre-ES5+ Idiomen beschäftigt und aufgezeigt, wie wir modernes JavaScript nutzen können, um H5P-Bibliotheken zu entwickeln. Hierfür wurden Buildprozesse vorgestellt, die modernes JavaScript in ein von H5P erwartetes, kompatibles Format überführen. Zudem wurde ein empfohlenes Setup sowie Templates für die Entwicklungsumgebung präsentiert.

Es wurden Lösungen recherchiert, die genutzt werden können, um Code Kompilierung und Interpretation im Client zu ermöglichen. Im Rahmen der Arbeit konnte ein erstes Konzept für die Auswertung von Code nicht-browser-nativer Programmiersprachen entwickelt werden. Hieran können zukünftige Arbeiten anknüpfen, um die vorgestellten Konzepte und Lösungen genauer zu evaluieren und sie gegebenenfalls in Prototypen oder MVPs umzusetzen.

Literatur

- [AG25] sr.solutions AG. *H5P Plugin for ILIAS*. 2025. URL: <https://github.com/srsolutionsag/H5P> (besucht am 16.05.2025).
- [AS25a] Joubel AS. *H5P Blanks Library*. 2025. URL: <https://github.com/h5p/h5p-blanks> (besucht am 27.05.2025).
- [AS25b] Joubel AS. *H5P Question Library*. 2025. URL: <https://github.com/h5p/h5p-question> (besucht am 27.05.2025).
- [Bab25a] Babel. *Config Files*. 2025. URL: <https://babeljs.io/docs/config-files> (besucht am 31.05.2025).
- [Bab25b] Babel. *Presets*. 2025. URL: <https://babeljs.io/docs/presets> (besucht am 31.05.2025).
- [Bab25c] Babel. *Transform Classes*. 2025. URL: <https://babeljs.io/docs/babel-plugin-transform-classes> (besucht am 31.05.2025).
- [Bab25d] Babel. *What is Babel?* 2025. URL: <https://babeljs.io/docs/> (besucht am 31.05.2025).
- [DiG25] James DiGioia. *babel-plugin-prismjs*. 2025. URL: <https://github.com/mAAdhaTTah/babel-plugin-prismjs> (besucht am 02.05.2025).
- [Doc25a] MDN Web Docs. *ECMAScript 6 Module Imports*. 2025. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules#Importing_modules (besucht am 26.05.2025).
- [Doc25b] MDN Web Docs. *IIFE*. 2025. URL: <https://developer.mozilla.org/en-US/docs/Glossary/IIFE> (besucht am 27.05.2025).
- [Doc25c] MDN Web Docs. *Import statement*. 2025. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import> (besucht am 26.05.2025).

- [Goo25] Google. *V8 JavaScript Engine*. 2025. URL: <https://v8.dev/> (besucht am 02.05.2025).
- [Gra25] GraalVM. *GraalVM WebAssembly Demo*. 2025. URL: <https://graalvm.github.io/graalvm-demos/native-image/wasm-javac/> (besucht am 02.05.2025).
- [H5P17] H5P. *Why develop for H5P?* 2017. URL: <https://h5p.org/node/3150> (besucht am 16.05.2025).
- [H5P25a] H5P. *command.md*. 2025. URL: <https://github.com/h5p/h5p-cli/blob/master/assets/docs/commands.md> (besucht am 23.05.2025).
- [H5P25b] H5P. *H5P Coding Guidelines*. 2025. URL: <https://h5p.org/documentation/for-developers/coding-guidelines> (besucht am 27.05.2025).
- [H5P25c] H5P. *H5P Command Line Interface*. 2025. URL: <https://h5p.org/h5p-cli-guide> (besucht am 24.05.2025).
- [H5P25d] H5P. *H5P File Format*. 2025. URL: <https://h5p.org/library-definition> (besucht am 24.05.2025).
- [H5P25e] H5P. *H5P Hello World Example*. 2025. URL: <https://h5p.org/tutorial-greeting-card> (besucht am 26.05.2025).
- [H5P25f] H5P. *H5P Semantic Versioning*. 2025. URL: <https://h5p.org/semantics> (besucht am 26.05.2025).
- [H5P25g] H5P. *H5P Technical Overview*. 2025. URL: <https://h5p.org/technical-overview> (besucht am 23.05.2025).
- [H5P25h] H5P. *The .h5p Specification*. 2025. URL: <https://h5p.org/documentation/developers/h5p-specification> (besucht am 24.05.2025).
- [JV14] Emery D. Berger John Vilks. *DOPPIO: Breaking the Browser Language Barrier*. Techn. Ber. University of Massachusetts, Amherst, 2014. URL: <https://plasma.umass.org/doppio-demo/paper.pdf>.
- [npm25] npm. *cross-env*. 2025. URL: <https://www.npmjs.com/package/cross-env> (besucht am 31.05.2025).
- [Pri25] Prism.js. *Prism.js*. 2025. URL: <https://prismjs.com/> (besucht am 02.05.2025).
- [Tug24] Sviatoslav Tugeev. *H5P tutorial*. 2024. URL: https://github.com/SvTHI/H5P_Development_Tutorial (besucht am 24.05.2025).
- [use25] Can I use. *Can I use... ECMAScript 6*. 2025. URL: <https://caniuse.com/es6> (besucht am 26.05.2025).

- [Web25a] Webpack. *babel-loader*. 2025. URL: <https://webpack.js.org/loaders/babel-loader/> (besucht am 31.05.2025).
- [Web25b] Webpack. *Configuration*. 2025. URL: <https://webpack.js.org/guides/getting-started/#using-a-configuration> (besucht am 31.05.2025).
- [Web25c] Webpack. *Installation*. 2025. URL: <https://webpack.js.org/guides/installation/> (besucht am 31.05.2025).
- [Web25d] Webpack. *Loaders*. 2025. URL: <https://webpack.js.org/concepts/loaders/> (besucht am 31.05.2025).
- [Web25e] Webpack. *Tree Shaking*. 2025. URL: <https://webpack.js.org/guides/tree-shaking/> (besucht am 31.05.2025).
- [Web25f] Webpack. *Webpack Concepts*. 2025. URL: <https://webpack.js.org/concepts/> (besucht am 31.05.2025).

Ehrenwörtliche Erklärung

Hiermit versichere ich, die vorliegende Bachelorarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben. Alle Stellen, die aus den Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Düsseldorf, 05. Juni 2025

Mohammed Schergis

Erklärung zur Nutzung generativer KI

Im Rahmen der vorliegenden Bachelorarbeit wurde generative KI zu folgenden Zwecken genutzt:

- DeepL Write zur sprachlichen Überarbeitung und Grammatikprüfung von Texten.
- ChatGPT um einen Textvorschlag zum Abstract zu generieren.
- Copilot während der Arbeit am Code und der Thesis als integriertes Werkzeug in der IDE.